Evaluación del módulo #6, #7 y #8

Consigna del proyecto 👨🏻‍💻

alkemy

Evaluación del módulo

Proyecto: Node & Express Web App

Situación inicial 📍

Unidad solicitante: Departamento de Desarrollo Backend de una empresa de tecnología.

Una reconocida empresa tecnológica necesita **desarrollar una aplicación web** para **gestión de usuarios y datos**. El equipo de arquitectura ha definido las bases del proyecto, pero requiere un **desarrollador backend** para llevarlo a producción.

Has sido seleccionado para este desafío. Tu misión será construir, iterar y demostrar el funcionamiento de una aplicación real utilizando **Node.js** y **Express**, aplicando conocimientos adquiridos en los **módulos 6, 7 y 8** del programa.

El proyecto deberá abordar, en **tres etapas**, los aspectos clave del desarrollo backend moderno: desde servir contenido web y gestionar rutas, hasta conectar con una base de datos, implementar un ORM, proteger rutas con JWT y exponer una API RESTful profesional.

Nuestro objetivo 📋

Desarrollar de manera individual una **aplicación web funcional** con **Node.js** y **Express**, que:

- Sirva contenido web dinámico.
- Gestione usuarios y datos persistidos en base de datos.
- Exponga una API RESTful con rutas protegidas mediante **JWT**.
- Integre persistencia con archivos planos, ORM para manejo de relaciones y funcionalidades clave como subida de archivos.

La implementación será **progresiva**, en tres partes alineadas a los módulos del curso:

→ **Módulo 6**: estructura inicial del servidor, rutas, vistas y persistencia básica.

→ **Módulo 7**: integración con base de datos, modelos y operaciones CRUD usando ORM.

→ **Módulo 8**: diseño de una API RESTful segura, con subida de archivos y autenticación.

Requerimientos ✅

Estos son los fundamentos que deberás respetar y aplicar durante todo el desarrollo del proyecto. Funcionan como condiciones base:

- La aplicación deberá estar desarrollada en **Node.js** (v18 o superior) utilizando el framework **Express.js**.
- El proyecto debe estar estructurado en **rutas**, **controladores**, **middlewares** y **servicios** siguiendo una arquitectura modular.
- Se espera el uso de **Sequelize** o **Mongoose** como ORM, según el tipo de base de datos seleccionada.
- La persistencia debe implementarse sobre una base de datos relacional (**PostgreSQL**) o documental (**MongoDB**).
- El repositorio del proyecto debe estar subido a **GitHub**, con estructura clara y un archivo `README.md` que documente su uso e instalación.
- Se deberá incluir al menos un mecanismo básico de seguridad utilizando **JSON Web Tokens (JWT)**.

Requerimientos específicos

Estos son los **elementos concretos** que la aplicación desarrollada deberá contemplar **funcional y técnicamente**. Serán validados en cada una de las entregas.

→ Gestión general de la aplicación

- Servir contenido web estático o dinámico desde **Express.**
- Implementar **rutas públicas y privadas** (protegidas con JWT).
- Permitir **autenticación de usuarios** (login / registro).
- Aplicar **lógica de persistencia** en archivos planos para funcionalidades simples (logs, configuración, etc.).
- Permitir **ejecución del servidor desde línea de comandos** con nodemon o scripts.

→ Acceso y gestión de datos

- Conectar la aplicación a una **base de datos real** (MongoDB o PostgreSQL).
- **Modelar y relacionar entidades** utilizando un ORM (relaciones 1:1, 1:N y N:M).
- Permitir **operaciones CRUD completas** sobre al menos dos entidades clave.
- Implementar **consultas filtradas y búsquedas** dinámicas.
- Aplicar **manejo de errores y validaciones** en las operaciones con base de datos.

→ Seguridad y API

- Crear una **API RESTful** con rutas protegidas mediante autenticación JWT.
- Implementar **subida de archivos** (por ejemplo, imágenes de usuario).
- Validar **tipos de archivo y tamaño** en el endpoint de subida.
- Estructurar las respuestas de la API con formato consistente (`status, message, data`).

Paso a paso 👣

Parte 3 – Módulo 8: Implementación de API Backend Node Express

Tu aplicación ya está funcionando localmente, con conexión a base de datos y lógica de negocio bien definida. Ahora, es momento de exponer esa funcionalidad mediante una API RESTful, de manera que pueda ser consumida por otras interfaces o clientes. Esta etapa es clave para consolidar tu perfil como desarrollador/a backend, ya que deberás integrar autenticación, subida de archivos y diseño robusto de endpoints, garantizando seguridad, claridad y escalabilidad.

Es por esto que en esta etapa final el objetivo será implementar una API RESTful segura, bien estructurada y documentada, que permita **consumir toda la lógica de la aplicación desde un cliente externo**. Se espera la integración de autenticación con JWT, manejo de archivos y definición clara de rutas y recursos.

1. **APIs RESTful** (Lección 1)

   - Objetivo específico**:** Comprender y aplicar los principios del diseño RESTful.

   - Tareas:

     → Diseñar y esquematizar los endpoints principales de tu app.
     → Aplicar convenciones REST en nombres, métodos y rutas.
     → Usar herramientas como Postman o Swagger para testear la estructura.

   - Requerimientos mínimos:

     → Al menos 4 endpoints bien definidos (`/usuarios`, `/productos`, etc.)
     → Métodos `GET`, `POST`, `PUT`, `DELETE` aplicados correctamente.

   - Tarea PLUS (opcional):

     → Documentar la API con Swagger/OpenAPI.

2. **Implementando una API REST** (Lección 2)

   - Objetivo específico**:** Construir e integrar los endpoints REST a tu aplicación.

   - Tareas**:**

     → Crear controladores que manejen la lógica de cada recurso.
     → Validar inputs y devolver errores controlados.
     → Organizar las rutas en archivos separados.

   - Requerimientos mínimos:

     → Endpoints funcionales y conectados a DB.
     → Estructura `controllers/, routes/, middlewares/.`

   - Justificación esperada:

     → ¿Cómo decidiste separar tus rutas y controladores?
     → ¿Qué validaciones realizaste antes de insertar/modificar datos?

3. **Subida de archivos al servidor** (Lección 3)

   - Objetivo específico: Integrar la funcionalidad de carga de archivos en el backend.

   - Tareas:

     → Instalar y configurar `multer` o herramienta equivalente.
     → Crear endpoint `POST /upload` que reciba un archivo.
     → Guardar los archivos en una carpeta pública.

   - Requerimientos mínimos**:**

     → Subida funcional de archivos con tipo controlado.
     → Carpeta `uploads/` organizada y accesible.

   - Tarea PLUS (opcional):

     → Asociar los archivos subidos a un registro en la base de datos (por ejemplo, foto de usuario).

4. **Securización mediante JWT** (Lección 4)

   - Objetivo específico: Proteger rutas y operaciones mediante autenticación basada en tokens.
   - Tareas:
     → Crear login (`POST /login`) que genere un JWT válido.
     → Proteger al menos 2 rutas con `middleware` de validación de token.
     → Verificar expiración y validez del token.
   - Requerimientos mínimos**:**
     → Al menos 2 rutas protegidas que no funcionen sin token.
     → Instrucciones claras de cómo autenticarse.
   - Justificación esperada:
   - ¿Por qué decidiste proteger esas rutas?
   - ¿Dónde y cómo almacenas el token?

✅ Entregables

1. **Repositorio actualizado**
   - Nuevas rutas, controladores y middlewares.
   - Estructura clara y finalizada del backend.
   - Archivo `README.md` con instrucciones de autenticación y endpoints.
2. **Subcarpeta en Drive:** *Parte 3 – Módulo 8*
   - Capturas de tests realizados con Postman o similar.
   - Evidencias de rutas protegidas, respuestas con error y éxito.
   - (Opcional) Captura de subida de archivos y su vinculación con la base de datos.
3. **Iteraciones y mejoras**
   - Si hiciste ajustes en partes anteriores (estructura, modelos, validaciones), incluye una breve nota o commit que explique el cambio.

🔍 ¿Qué vamos a validar?

En esta entrega final del proyecto integrador, se evaluará que la aplicación esté expuesta como una **API RESTful funcional, segura y coherente**. Además, se validará que el desarrollo haya **integrado mejoras, iteraciones o ajustes** sobre las partes anteriores, consolidando un producto backend real.

- ★ Aplicación de requerimientos técnicos

  - ☐ Implementación completa de endpoints REST con métodos y rutas adecuadas.
  - ☐ Correcta integración de `controllers, routes, middlewares`.
  - ☐ Subida de archivos con configuración adecuada (`multer` o similar).
  - ☐ Protección de rutas con JWT y validación de tokens.
  - ☐ Aplicación robusta, sin errores y conectada a base de datos.

  Se evaluará tanto la nueva funcionalidad como la consolidación del backend general.

- ★ Objetivo de la consigna

  - ☐ La API resuelve correctamente las operaciones de la app.
  - ☐ Se evidencia una arquitectura pensada para escalabilidad.
  - ☐ La app queda en condiciones de ser consumida desde un cliente externo (front-end, mobile, etc.).

  Se valora que el entregable cierre el ciclo de backend: estructura, base de datos, lógica y API.

- ★ Justificación y reflexión

  - ☐ Documentación en README sobre uso de rutas, autenticación y carga de archivos.
  - ☐ Justificación técnica de decisiones sobre seguridad, estructura de rutas, asociación de archivos, etc.
  - ☐ Reflexión sobre cómo se integraron los aprendizajes de los 3 módulos.

  Esta etapa también valida tu capacidad de argumentar cómo iteraste y mejoraste entregas anteriores.

★ Claridad y organización

- ☐ Código bien estructurado, nombrado y modularizado.
- ☐ `README.md` con explicación clara de cómo consumir la API.
- ☐ Entrega ordenada en GitHub y en la carpeta compartida.

💼 Portafolio

Esta parte consolida todo tu backend en una API RESTful profesional. De cara al portafolio, representa una muestra concreta de tu capacidad para:

- Diseñar y exponer una API funcional y segura.
- Aplicar autenticación moderna con JWT.
- Trabajar con manejo de archivos, modularidad, y buenas prácticas de desarrollo backend.

¡Éxitos!

Nos vemos más adelante

alkemy